OppRadar

SECURITY DOCUMENTATION

Security review (STRIDE)

Threat-model review of the web app, the API, the Supabase schema and how the API uses the AI package.

From: OppRadar Team
Date: 25 October 2026

Hi,

I ran a STRIDE threat-model review of OppRadar before we deploy. It covers the web app, the API, the Supabase schema and how the API uses the AI package. This report covers what I found, what is already fixed, and, in section 2, **what I need from you on the AI package**.

1 Where things stand

The foundations held up well:

- Every API request is checked against a verified Supabase JWT.
- Only Supabase can grant the admin role.
- Row-level security is on for every table, and a test enforces it.
- The service-role key never reaches the browser.

> The review found **11 issues**:
> - **5 are fixed** and covered by tests (`backend/tests/test_security.py`).
> - **2 are configuration tasks** for deployment.
> - **4 are low risk and accepted for launch**, each with a follow-up.

The most serious one (F1) needed a database migration. It is **applied and verified on our Supabase project**: direct writes are refused, and saving through the API still works.

After the fixes, I ran the full signed-in journey against Supabase: sign-up, the 5 setup steps, first AI scan, saving, applying, Premium, and deleting the account. **It passes.** That run also surfaced four bugs in how the API handles the AI pipeline. They are fixed and listed in section 5, since some of them touch your package.

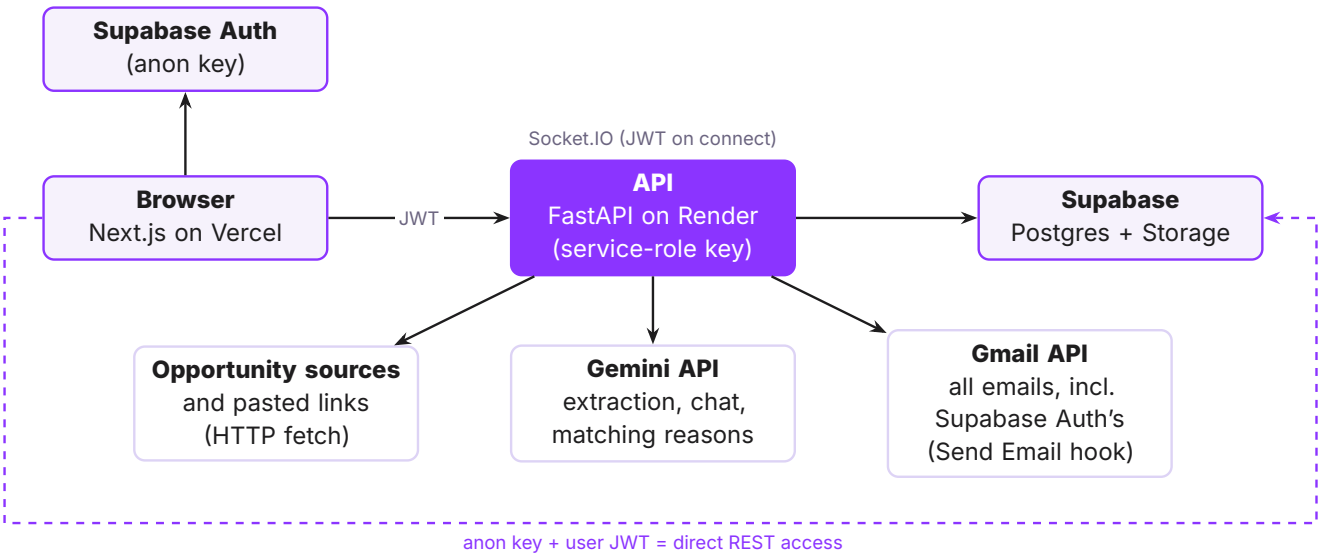
#	Finding	STRIDE	Risk	Status
F1	Users could write their own rows directly through Supabase's REST API, skipping the API's checks, including on the shared demo account	T, E	HIGH	Fixed: migration `0009`, applied and verified
F2	Links pasted in chat were fetched by the server without checking where they point (SSRF)	I, E	MEDIUM	Fixed in the API. **Redirects still need a fix in the AI package (section 2)**
F3	The rate-limit client IP could be spoofed with an `X-Forwarded-For` header	D	MEDIUM	Fixed; `CLIENT_IP_HEADER` gets set at deploy
F4	A 5 MB PDF with thousands of pages would be OCR'd page by page on our single instance	D	MEDIUM	Fixed: 10-page limit on uploads
F5	Chat and application history were never deleted, although `/privacy` promises it	I (privacy)	MEDIUM	Fixed: retention purge in the scheduled cycle
F6	The unsubscribe-link secret falls back to other secrets	S	LOW	Config: set `EMAIL_SECRET`
F7	No DMARC record for the sending domain	S	LOW	Config: DNS at deploy

#	Finding	STRIDE	Risk	Status
F8	Ingested opportunities are published automatically, so scam or injected pages reach users labelled, not blocked	T	LOW	Accepted, follow-up (touches ingestion)
F9	Admin edits and deletes leave no audit trail	R	LOW	Accepted, follow-up
F10	AI errors return internal detail (`str(exc)`) to the client	I	LOW	Accepted, follow-up
F11	No `script-src` Content Security Policy	T, E	LOW	Accepted, follow-up

2 What I need from you (AI package)

1. **`PageFetcher` follows redirects without re-checking them (F2).** The API now refuses pasted links that point at private, loopback or metadata addresses (`backend/app/netguard.py`). But `ingestion/fetch.py` uses `follow_redirects=True`, so a public URL that redirects to `http://127.0.0.1/...` is still fetched.
   - Could you check each redirect hop's address, or turn off automatic redirects and follow them yourself with the same check?
   - Resolving and checking per hop would also close the DNS-rebinding gap.
   - The API's check is a single function, `is_public_url(url)`, if you want to reuse the logic.
2. **Embedding model: `bge-m3` does not fit our server.**
   - The default model is a 2.2 GB download and needs about as much RAM, which is too much for a ~1 GB Render instance.
   - Locally the download filled the disk, then a memory allocation failure (a Rust panic in the model loader) killed the whole process. Because it's a crash, not a Python exception, the `embedding_fallback_model` code never ran.
   - I am deploying with `EMBEDDING_MODEL=sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` (your fallback: about 470 MB, 384-dim, covers AR/FR/EN). Live matching works with it: scores and reasons look reasonable on the test profiles.
   - **Please confirm the quality trade-off is acceptable**, or suggest another model under about 500 MB.
3. **Model loading is not thread-safe.**
   - `get_embedder()` shares one `Embedder`, but `_ensure_model()` has no lock, so two scans starting together loaded two copies (twice the memory).
   - On the API side, `services/ai.py` now loads the model once at start-up and holds a lock while it does. A lock inside `_ensure_model()` would fix it for every caller.
4. **FYI: criteria status.** Your `CriteriaStatus` uses `missed`. Our API had `unmet`, so the feed crashed when a requirement was missed. The API now maps `missed` to `unmet` when it reads matches, so no change is needed on your side. Just keep the values as they are, or tell me if you change them.
5. **Optional (F8): auto-published ingestion.** New opportunities are stored as `approved`.
   - If the scam check returns `suspicious` or `scam`, I'd like to send the item to the admin review queue instead of publishing it.
   - Is the trust label reliable enough to route on, or should I also look at specific `scam_flags`?

3 The system and its trust boundaries



Assets

- Students' profiles: personal data, CVs and goals.
- Their consents.
- The integrity of published opportunities, because students act on them.
- The admin role.
- The Gemini budget and the Gmail sending account (and its daily sending limit).
- The service-role key, which bypasses RLS.

Trust boundaries

1. The browser and the API.
2. The browser and Supabase directly (the anon key is public by design).
3. The API and third-party content: source pages, pasted links and pasted text, all of which reach the LLM.
4. The API and the services it calls: Gemini, Gmail and Supabase.
5. Supabase Auth and the API's Send Email hook (`POST /email/auth-hook`, signed by Supabase).

4 Findings by STRIDE category

S Spoofing

Already in place:

- **JWT checks.** The API verifies the JWT signature (ES256/RS256 via JWKS, or HS256) and its audience on every request (`backend/app/auth.py`).

- **Socket.IO.** Connections need a valid token and join only their own `user:<id>` room.
- **Unsubscribe links.** They are HMAC-SHA256-signed and compared in constant time.
- **Demo password.** It never leaves the server; the browser only gets a session.

F6 Unsubscribe secret fallback LOW

Without `EMAIL_SECRET`, the signing key falls back to the JWT secret, then the service-role key, then `"dev"`.

- *Fix:* a dedicated random `EMAIL_SECRET` in production.

F7 Email spoofing LOW

We now send through the Gmail API, from a Gmail account.

- With an `@gmail.com` address, Google publishes SPF, DKIM and DMARC for the domain: nothing to do.
- With our own domain (Google Workspace), set up DKIM in the Workspace admin console and add a DMARC record (`p=none` first, then `quarantine`).
- **The auth email hook** checks Supabase's Standard Webhooks signature and refuses calls older than 5 minutes, so nobody else can make the API send confirmation or reset emails (`backend/tests/test_gmail.py`).

T Tampering

Already in place:

- **Validated writes.** All writes go through the API, which validates them with Pydantic.
- **Read-only demo.** The `Writer` dependency blocks the demo account.
- **Reviewed submissions.** Organisation submissions arrive as `pending`.
- **Escaped output.** React escapes all text, AI output included, and AI text in emails is HTML-escaped.

F1 Direct REST writes bypassed the API HIGH FIXED & APPLIED

- **The problem:** migrations 0001–0006 gave signed-in users `for all` policies on their own rows (profile, skills, documents, chat, applications, consents, storage files). The app never used them, but anyone could pair the public anon key with their own token and write through Supabase's REST API, skipping every API check.
- **Worst case:** the demo session is shared by every visitor, so anyone could deface the demo profile. They could also forge rows in the `consents` audit log.
- **Fix:** `supabase/migrations/0009_read_only_client_access.sql` keeps read access to one's own rows and removes all client writes. The API uses the service role, so its behaviour is unchanged.
- **Verified live:** writes through the API succeed. Direct updates and inserts on profiles, skills, consents and chat, and direct CV uploads to storage, are all refused.

F8 Auto-published ingestion LOW ACCEPTED

Ingested opportunities default to `approved`.

- **Mitigations:**
  - Scam detection sets `trust_level` and `scam_flags`, and the UI labels them.
  - The LLM only fills a fixed schema.
  - Users are always sent to the official page.
- *Follow-up:* see section 2, point 5.

F11 No script-src CSP LOW ACCEPTED

The app sends `frame-ancestors`, `base-uri`, `form-action`, `object-src`, `nosniff` and HSTS. A `script-src` policy needs nonces for Next.js's inline scripts.

- *Follow-up:* add one after launch.

R Repudiation

Already in place:

- **Consents** are an append-only log.
- **Reviews** record `reviewed_by` and `reviewed_at`.
- **Ingestion runs** record `triggered_by`.
- **Sent emails** are stamped `emailed_at`.

F9 No admin audit trail LOW ACCEPTED

Edits and deletes in `/admin` record nothing about who made them.

- *Follow-up:* an `admin_audit` table.

I Information disclosure

Already in place:

- **Owner-only access.** RLS and the API both scope data to its owner.
- **CVs** sit in a private bucket under `<user_id>/`.
- **Consent-gated AI.** The LLM reads CVs only with the CV consent, and profiles are scored only with the matching consent.
- **Keys and errors.** Secrets exist only on the server, and the 500 handler returns a generic message.
- **Account deletion** removes the auth user, the files and every row.

F2 SSRF through pasted links MEDIUM FIXED IN THE API

- **The problem:** a link pasted in chat is fetched so the AI can read the announcement, and nothing checked its destination. `http://127.0.0.1:8000/...`, private hosts or cloud metadata addresses could have been read from inside our network, and their text returned through the extracted fields.
- **Fix:** `netguard.is_public_url()` allows only http(s) on the default ports, where every resolved address is public.
- **Remaining:** redirects (section 2, point 1).

F5 Retention not enforced MEDIUM FIXED

- **Fix:** `agent.purge()` runs after every scheduled cycle. It deletes chat older than 90 days, and applications untouched for 12 months for users on the "12 months" setting.

F10 AI error detail LOW ACCEPTED

The `AIError` handler returns `str(exc)`.

- *Follow-up:* return a fixed message for each `code` and keep the detail in the logs.

D Denial of service

Already in place:

- **AI budgets.** Manual ingestion has a 15-minute cooldown, and each user gets 20 AI chat answers a day.
- **Ingestion bounds.** Per-run limits on extractions and detail fetches, plus timeouts.
- **One scheduler**, guarded by a cycle lock.
- **Uploads** are capped at 5 MB with a PDF signature check.

F3 Spoofable rate-limit IP MEDIUM FIXED

- **The problem:** I had started uvicorn with `--forwarded-allow-ips "*"`, which makes it trust the leftmost `X-Forwarded-For` entry. The client controls that entry, so the per-IP limits on `/auth/demo`, `/opportunities` and `/chat` could be bypassed.
- **Fix:** `ratelimit.client_ip()` reads one configured source, `CLIENT_IP_HEADER`. For `x-forwarded-for` it takes the last entry, the one the platform appends.
- **At deploy:** set it using **Admin → Ingest → Client IP check** from two different networks.

F4 PDF page bomb MEDIUM FIXED

OCR runs at 220 dpi per page, and the limit was in bytes only.

- **Fix:** uploads over 10 pages are refused (`UPLOAD_MAX_PAGES`).

Accepted limits:

- **In-memory counters.** Rate limits, the cooldown and the chat quota live in memory, which is correct only on one instance. Scaling out needs Redis.
- **Shared demo quota.** All demo visitors share one daily AI quota.

E Elevation of privilege

Already in place:

- **Admin role.** It comes from `app_metadata.role`, which only the service role can set.
- **Two layers of admin checks.** Admin routes need `require_admin`, and the database checks `private.is_admin()`.
- **Read-only demo.** Enforced in the API and, since 0009, in the database too.
- **CORS** is limited to our origin.
- **No CSRF exposure:** the API uses bearer tokens, not cookies.
- **No open redirect:** auth redirects only accept same-origin paths.

Elevation also appears in F1 (getting around the read-only demo) and F2 (using the server's network position).

5 Reliability bugs found by the live run (all fixed)

These are not security findings, but the live journey against Supabase turned them up, and three of them involve the AI pipeline:

Bug	Fix
After a restart, the first user's scan waited about 80 s for the embedding model to load	The model loads in the background at start-up (about 18 s here)
Two scans at once loaded the model twice	A lock in the API gateway (section 2, point 3)
Two scans of the same user collided on `matches`' unique key (500)	One scan per user at a time
The feed crashed on the AI's `missed` criteria status (500)	Mapped to `unmet` (section 2, point 4)
A Supabase timeout (HTTP 522) surfaced as a 500	The API now answers 503 "please try again" with `Retry-After`

6 Remaining before launch

1. **Production environment variables:**
   - `EMAIL_SECRET` (random, 32+ bytes).
   - `CLIENT_IP_HEADER` (checked with the admin tool).
   - `CORS_ORIGINS` (the production domain only).
   - `AGENT_SCHEDULER=1` on the single instance.
   - `EMBEDDING_MODEL` (pending your answer on section 2, point 2).
2. **Migration 0009 on the production Supabase project.** It is already applied on dev.
3. **Email:** set up the Gmail sender (`GMAIL_*`, `MAIL_FROM`, `SUPABASE_EMAIL_HOOK_SECRET`). DNS only if we send from our own domain (F7).
4. **Supabase Auth:**
   - Turn on leaked-password protection.
   - Turn on email confirmation, and point the Send Email hook at `https://<api>/email/auth-hook`.
   - Keep only the production redirect URLs.
5. **Your side:** section 2, points 1–3.
6. **Legal review** of the terms and privacy drafts.

After launch: F8, F9, F10, F11, and a dependency scan in CI (`pip-audit`, `npm audit`). I'll re-run this review when Telegram alerts or Premium payments add new trust boundaries.

Thanks! Let me know if anything here is unclear or you see it differently.